

XXXXX XXXXXXXXXXXX^{a,*1}, *****^a, *****^a, *****^b, *****^a,
*****^a and *****^a

a *****
b *****

Keywords:
benchmark
dataset
Text-to-SQL
SQLite
LLM
Digital Forensics
Mobile Forensics

SQLite databases play a central role in mobile phone forensics. Mobile applications frequently use them for data storage. Efficient extraction and interpretation of SQLite data is crucial for reconstructing device usage and user activities. In practice, digital forensic investigators must formulate and analyse complex SQL queries to retrieve evidence from various heterogeneous databases. This task requires extensive expertise in SQL, database schemas, and application-specific data logic.

In this paper, we investigate an LLM-based approach to assist digital forensic investigators by automating the generation of SQL queries for forensic analysis. This enables investigators to query SQLite databases more efficiently and with less technical effort. First, we propose a mobile forensic dataset that captures typical investigative questions and database structures. We then use this model to fine-tune a local LLM. We introduce ForSQLiteLM, a 4-bit quantised LLaMA-3.1-8B model. It is optimised on a self-defined, domain-specific dataset tailored to mobile forensic scenarios. We compare ForSQLiteLM with several state-of-the-art LLMs to evaluate its effectiveness in generating forensic queries. We show that effective forensic Text-to-SQL generation can be achieved with a locally deployable 3B-parameter LLM by combining realistic SQLite schemas, execution-based evaluation, and domain-specific fine-tuning. Finally, as a proof of concept, we demonstrate how the proposed model can be integrated into the FQLite data retrieval tool via a retrieval-augmented generation (RAG) pipeline.

Mobile phone forensics is a crucial aspect of modern digital investigations. Smartphones continuously collect and store extensive data about user behaviour. Much of this information is stored in application-specific databases, most of which are implemented with SQLite. These databases contain valuable forensic artefacts such as communication histories, browsing activities, location data, system logs and records of application usage. Forensic examination of mobile databases reconstructs user actions, builds timelines, and reveals key evidence ?.

Despite its importance for evidence gathering, analysing mobile databases remains a technically challenging and error-prone task. Investigators must understand various database schemas, application-specific data models, and complex SQL constructs in order to retrieve relevant evidence. This process not only requires advanced technical expertise but also carries a significant risk of incomplete or incorrect query formulations. As mobile ecosystems continue to grow and application databases increase in number and complexity, database analysis is becoming an increasingly significant bottleneck in mobile forensic workflows.

To address these challenges, this article examines the extent to which domain-specific large language models can effectively support mobile forensic database analysis. The

focus is on the automated generation of SQL queries from natural language questions. In particular, we investigate the extent to which a locally fine-tuned LLM can reduce the technical burden on investigators while maintaining the accuracy and reliability required in forensic contexts. Standard large language models (LLMs) often deliver only mediocre results when tasked with generating SQL statements. These models tend to struggle to create accurate SQL queries, even when provided with the database schema and table relationships. This limitation can hinder the efficiency of forensic analysis.

This work introduces *ForSQLiteLM*, a domain-adapted, locally deployable Text-to-SQL model designed for mobile phone forensics. The model was fine-tuned on a comprehensive forensic dataset comprising authentic SQLite schemas extracted from real forensic tools, as well as execution-validated SQL queries covering a wide range of artefacts from mobile devices, systems, and third-party applications. To ensure practical applicability in privacy-sensitive investigations, the model is trained and evaluated entirely in a local environment without relying on external cloud services.

Before presenting our approach, we review relevant work. This article focuses on two areas of research: forensic analysis of SOLite databases and Text-to-SQL generation.

There are numerous publications on SQLite forensics ???. The National Institute of Standards and Technology (NIST)

*Corresponding author

*****@*****.** (X. XXXXXXXXXXXX)

 www.foobar.** (X. XXXXXXXXXXXX)

ORCID(s): 0000-0001-7485-xxxx (x. xxxxxxxxxxxx)

has published an official test guide for SQLite data recovery tools as part of the Computer Forensics Tool Testing (CFTT) program [2]. All of the mentioned publications on SQLite forensics primarily focus on recovering deleted data. However, evaluation using SQL statements is not discussed. However, the interpretation of data, for example by combining different databases to make the data analyzable, is not discussed.

Much of the current knowledge about forensic SQLite analysis comes from practical sources like forensic tool vendors, technical blogs, and professional forum, rather than academic journals. If such data is not available, the only option is the time-consuming reconstruction of database schemas through reverse engineering. While unconventional for academic research, these sources are valuable because mobile applications and their databases evolve faster than peer-reviewed publications [2]. Practitioners performing actual forensic investigations often document new database structures first, before academic studies appear. This approach is methodologically sound when practical sources are critically evaluated, transparently cited, and combined with established scientific literature. Nevertheless, in court, it can be critical in terms of a forensic approach if the methodology used is not scientifically substantiated.

Text-to-SQL Generation

Within Text-to-SQL area research aims to translate natural language questions into executable SQL queries. Early work introduced large-scale datasets like WikiSQL [2], which demonstrated feasibility but was limited to single-table schemas. Spider [2] addressed these limitations with cross-domain, multi-table schemas and complex SQL constructs. Building on Spider, RAT-SQL [2] integrated relation-aware modeling of schema structure, while PICARD [2] enforced grammar and schema constraints during generation.

Recently, large language models such as GPT-3 [2] and code-oriented variants have shown strong zero-shot and few-shot capabilities. The BIRD benchmark [2] extends this work by requiring models to reason over actual database contents and realistic, noisy data-considered more challenging than earlier benchmarks. Recent surveys [22] provide comprehensive overviews of current developments.

Despite extensive work in both areas, no publication addresses Text-to-SQL specifically for digital forensics. A recent study presented current applications of LLMs in digital forensics [2]. None of the work presented in it addresses the topic of Text-to-SQL. As of January 2026, the Hugging Face platform [2] - a leading repository for AI models - hosts over 4,200 Text-to-SQL models, yet none target SQLite specifically or the forensic domain. Only one model on the platform address digital forensics at all, namely the `Advanced_Forensic_Investigations_Knowledge_Library` and neither focuses on SQL generation. Existing Text-to-SQL approaches face three limitations for forensic applications:

1. **Training data mismatch:** Models are trained on business databases (e.g., Spider, BIRD) with standard

schemas, not mobile forensic databases with undocumented, application-specific structures.

2. **SQL dialect differences:** Training datasets primarily use MySQL or PostgreSQL syntax, not SQLite-specific constructs commonly found in mobile applications.
3. **Deployment constraints:** State-of-the-art models rely on cloud APIs (e.g., GPT-4), violating privacy requirements in forensic investigations where data must remain local.

In contrast to prior work, this paper focuses on forensic Text-to-SQL, emphasizing authentic SQLite schemas extracted from real forensic tools, execution-validated queries, and local deployment on investigative hardware. Rather than optimizing benchmark performance alone, we prioritize correctness, and practical usability in digital forensic workflows.

3. The "SQLiteDS" Dataset

A suitable dataset is essential prior to fine-tuning a large language model (LLM), as fine-tuning does not define the task itself but rather trains the model to perform a specific task correctly within a given domain. Without an appropriate dataset, fine-tuning can degrade performance rather than improve it.

For this reason, we first created a suitable dataset. The Forensic SQLite Dataset (SQLiteDS) is a comprehensive Text-to-SQL dataset specifically designed for digital forensics investigations. The dataset enables automated SQL query generation for extracting evidence from forensic databases across multiple platforms and artifact types. *Table 1* provides an overview of the composition and characteristics of the training dataset used in this study. The dataset comprises 687 base examples, which were extended to 744 examples through additional data enrichment, and covers a total of 148 distinct forensic categories. The examples span multiple platforms, including iOS, Android, Windows, web-based artifacts, and various third-party applications, ensuring broad applicability across common forensic scenarios.

To achieve a balanced evaluation, the dataset is evenly distributed across three difficulty levels - easy, medium, and hard - each accounting for approximately one third of the examples. On average, the SQL queries consist of 85 tokens, reflecting realistic query lengths encountered in practical investigations, while schema complexity ranges from 2 to 15 tables per database, capturing both simple and highly interconnected forensic data structures. Listing 1 shows an example of a training prompt from the dataset.

PROMPT: "Reconstruct Bluetooth device timeline for a specific date"

DIFFICULTY: "hard"

CONTEXT: "ios_bluetooth"

RESPONSE:

Table 1

Key characteristics of the forensic SQL dataset with forensic relevance.

Feature	Description	Forensic Value
Number of examples	774 forensic SQL queries covering realistic investigative scenarios	Provides sufficient coverage for model training and evaluation across diverse forensic tasks
Artifact categories	148 unique categories spanning multiple applications and systems	Ensures broad coverage of potential evidence sources encountered in investigations
Platform coverage	iOS, Android, Windows, Web, and third-party applications	Reflects the multi-platform nature of modern digital evidence, supporting cross-device analysis
Difficulty levels	3 levels (Easy, Medium, Hard), balanced at approximately 33% each	Allows assessment of model performance on queries of varying complexity, simulating real investigative challenges
Forensic domains	Messaging, web browsing, system logs, cloud storage, and cryptocurrency	Captures the most relevant domains for digital forensics, from communication data to financial traces
Schema authenticity	Real SQLite schemas extracted from actual forensic tools	Guarantees practical applicability and realistic query generation for investigators
Execution validity	All queries tested for correctness within their respective database schemas	Ensures reliability of results and supports reproducible analysis in forensic workflows

```

SELECT cl.device_address, d.name, DATETIME(cl.
  connection_time + 978307200, 'unixepoch') as
  connected, DATETIME(cl.disconnect_time +
  978307200, 'unixepoch') as disconnected, cl.
  connection_duration / 60.0 as minutes, cl.
  signal_strength
FROM connection_log cl
JOIN devices d ON cl.device_address = d.address

WHERE DATE(cl.connection_time + 978307200, 'unixepoch') =
  '2026-01-01'
ORDER BY cl.connection_time;

```

Listing 1: Example record from the forensics dataset

The dataset is designed to reflect the complexity and diversity of real-world mobile forensic investigations. It covers a wide range of SQLite database schemas with varying levels of structural complexity. Simple schemas consist of one to three tables and typically represent messaging applications or basic system logs. Medium-complexity schemas include four to seven tables and are commonly found in web browsers and general system databases. Complex schemas comprise up to fifteen interrelated tables, as observed in artifacts such as the iOS textitKnowledgeC database and the Windows timeline, requiring advanced query logic and multi-table joins. To support realistic investigative workflows, the dataset includes a broad spectrum of temporal queries. These range from relative time expressions (e.g., last 24 hours or past week) to absolute timestamps and date ranges, as well as time-based aggregations such as daily,

weekly, or monthly statistics. Such queries are essential for reconstructing user activity timelines and identifying temporal patterns in forensic evidence. In addition, the dataset explicitly models common forensic analysis patterns. These include timeline reconstruction through chronological ordering of events, relationship analysis to identify connections between entities such as users, devices, or applications, and pattern detection to uncover anomalous or suspicious behavior. Data correlation across multiple tables and frequency-based analyses are also well represented, reflecting typical investigative tasks such as counting interactions, communications, or application usage.

Given the sensitive nature of digital forensic data, the dataset incorporates numerous privacy-relevant query scenarios. These address areas such as location tracking, communication behavior, browsing history, financial transactions, and health related data. This design choice ensures that the dataset realistically captures the ethical and technical challenges faced by practitioners when analyzing personal and sensitive information.

Finally, the dataset leverages SQLite specific functionality to mirror practical forensic query requirements. It includes extensive use of date and time functions such as `strftime()`, `datetime()`, and `julianday()` for temporal reasoning, as well as JSON extraction via `json_extract()` and fulltext search capabilities using the `MATCH` operator.

Together, these features ensure that the dataset provides a comprehensive and technically realistic foundation for

Table 2
Statistics of the dataset used for training and evaluation.

Metric	Value
Total Examples	687 (base) / 744 (extended)
Unique Categories	148
Platform Coverage	iOS, Android, Windows, Web, Third-Party Apps
Difficulty Levels	Easy (33%), Medium (33%), Hard (33%)
Average Query Length	85 tokens
Schema Complexity	2–15 tables per schema

Table 3
Overview of dataset categories and domain coverage.

Domain	Subdomains (Examples)	Categories	Examples
Mobile Forensics (iOS)	Messaging, Browsers, Location, Media, System, Finance	29	~170
Mobile Forensics (Android)	Messaging, Google Services, Social Media, System, Location	30	~170
Windows Forensics	Event logs, Execution Artifacts, File System, Network, Timeline	18	~132
Third-Party Applications	Cloud, Social Media, Gaming, Finance, Travel, Health	60	~230
Multi-Domain Scenarios	E-Commerce, Healthcare, Banking, IoT, Logistics, Gaming	11	~72
Total	–	148	~774

training and evaluating LLM based approaches to automated SQL query generation in digital forensic contexts.

4. Model finetuning

Once the dataset had been defined, the next step was to select a suitable baseline model so that the actual training could begin.

The model used for fine-tuning must be easy to train and at the same time lean enough to be able to run on normal hardware. For privacy and data protection reasons, we opted to train a local language model on our forensic Text-to-SQL dataset. The selected model has 3 billion parameters and requires 4–6 GB of RAM and 6–8 GB of VRAM (reduced to 4 GB after quantization). This configuration provides several key advantages:

- **Runs on nearly all computers**, including systems with integrated GPUs, enabling accessible local training without specialized hardware.
- **Strong instruction-following capabilities**, ensuring accurate query generation based on natural language prompts.
- **Officially optimized for Text-to-SQL tasks**, aligning directly with our forensic use case.
- **Fast inference** (less than 1 second per query), supporting interactive analysis workflows.
- **Excellent cost-to-performance ratio**, offering high-quality results without requiring large-scale cloud infrastructure.

This approach allows the model to better understand the intricacies of the data and generate more precise SQL queries. By leveraging such a fine-tuned LLM, investigators can accelerate the process of database analysis and improve the

accuracy of their findings. The choice ultimately fell on the LLaMA-3.2-3B¹ model developed by the Meta Group.

Overall, this setup allows for efficient and secure fine-tuning of our forensic dataset on standard laptops or desktops, making it practical for research and investigative applications where sensitive data must remain local.

4.1. Training procedure

The training was adapted using parameter-efficient fine-tuning instead of complete retraining (Figure 1). Specifically, low-rank adaptation (LoRA) was used to modify the behavior of the model while retaining most of the original weights. Instead of updating all parameters of a large LLM, LoRA freezes the original, pre-trained model weights. It introduces small, customizable “adapter” matrices alongside the existing, fixed matrices. As a result, only around 2.5 percent of the weights were recalculated. This approach is particularly well suited to local environments, as it reduces the computational effort and limits the risk of catastrophic forgetting. In addition, this approach allows for clearer documentation of model changes.

The LoRA configuration was designed to balance expressiveness and efficiency. A rank of 16 with a scaling factor (alpha) of 32 was chosen, allowing the model to learn domain-specific patterns without introducing excessive additional parameters. The adaptation was applied to the attention projection layers, the query, key, value, and output projection modules. This is because these components are critical for controlling the integration and weighting of contextual information during inference.

Training was performed over five epochs with a learning rate of 10^{-5} . Due to hardware limitations and the desire to maintain the ability to process long contexts, a batch size of one was used in combination with gradient accumulation over four steps. The batch size determines how many training

¹<https://huggingface.co/meta-llama/LLaMA-3.2-3B>

examples are passed through the model at once before the model weights are updated. In our case this results in an effective batch size of four. A maximum sequence length of 2048 tokens was explicitly configured, as preserving long forensic narratives and structured evidence without truncation is critical for downstream RAG performance.

Optimization was performed using the AdamW optimizer, which is well suited for fine-tuning transformer-based models. It offers improved regularization and leads to better generalization ability of the model. A cosine learning rate schedule with a warm-up phase covering 10% of the total training steps was applied to ensure stable convergence, especially in the early stages of training. The learning rate starts at its maximum value (the target value of the warm-up phase) and then decreases gently (not abruptly as with stepwise reduction) in the form of a half cosine wave. Gentle annealing enables efficient exploration of the loss landscape at the beginning with a higher learning rate and fine tuning of the parameters towards the end with a lower learning rate.

The training was conducted over five epochs. Given the relatively small effective batch size and the domain-specific nature of the data, multiple passes through the dataset were required. Empirically, five epochs offer a good compromise between underfitting and overfitting: fewer epochs resulted in insufficient domain adaptation, while additional epochs did not bring measurable improvements and increased the risk of memorizing case-specific formulations.

We applied data augmentation to increase training set diversity and improve model generalization. Each of the 768 original examples was paraphrased three times generating semantically equivalent but syntactically varied natural language requests while preserving the SQL queries unchanged. This resulted in 2,304 training examples (3× expansion), helping the model learn to handle diverse phrasings of the same forensic request rather than memorizing specific question templates.

All experiments were performed on Apple M2 hardware using the Metal Performance Shader (MPS) backend. Under this configuration, the entire fine-tuning process took approximately 3.5 hours.

In the last step the fine-tuned model was converted to GGUF format for efficient deployment using llama.cpp ?. Quantization to 4-bit (Q4_K_M) reduced model size from 6GB to 2.3GB with minimal quality loss (~95% of FP16 performance), enabling deployment on consumer hardware and edge devices. Inference speed improved 2.5× compared to FP16, achieving 30-40 tokens/second on Mac Mini M2 CPU-only execution. That concluded the training step.

4.2. Benchmark

Now it was time to determine the benefits of the training. Therefore a benchmark was developed to measure the success. At the same time, a comparison with other LLMs was to be made.

For the comparison, we initially selected the baseline model we had trained on, in addition to our fine-tuned model. We also selected the 7B CodeLLaMa ?, which specializes

in coding, and the current GPT4o (Omni) ? as the gold standard. Of course, we could only test the latter online via API key. We tested the other systems locally.

To make the benchmark as realistic as possible, the schemas were transferred directly to an SQLite database. The SQL statements generated by the LLM were then executed directly on the SQLite database. This ensures syntactic correctness. The semantic correctness of the generated queries was also checked individually by experts.

As expected, GPT4o performed best as the top dog and took first place in our benchmark. In second place was our fine-tuned model achieves an overall execution accuracy of 79%, with performance strongly correlated to query complexity (Table 4). For easy, single-table queries, the model reaches 94% accuracy, demonstrating reliable handling of straightforward forensic questions (Table 5). Accuracy decreases to 81% for medium-complexity queries involving simple joins and to 62% for hard queries with complex join structures, nested conditions, or advanced aggregations. These results indicate that while the model generalizes well across common forensic scenarios, query complexity remains a key challenge.

The CodeLlama 7B achieved 55%. While its code specialization improves over base Llama (45%), lack of domain fine-tuning limits performance compared to our approach (79%). Error analysis shows that failures are predominantly caused by semantic errors (18%), rather than syntactic issues. The most frequent error source is missing or incorrect join conditions (8%), followed by table confusion (6%) and column confusion (4%). These error patterns suggest that the model's limitations primarily arise from difficulties in reasoning over complex schema relationships rather than from insufficient SQL syntax knowledge. In contrast, syntax errors are rare (3%), and schema generation errors are fully eliminated, confirming that the model consistently adheres to the provided database schemas.

From a forensic perspective, the higher error rate for complex queries highlights the need for improved schema reasoning and multi-table context awareness, particularly in databases with overlapping or semantically similar artifacts. Nevertheless, the model's strong performance on easy and medium queries indicates substantial potential to reduce investigator workload in routine database analysis tasks. Importantly, all queries were executed and validated against real SQLite schemas, ensuring that reported accuracy reflects practical usability rather than surface-level correctness. Several key improvements contributed to the observed performance gains. Compared to the baseline model the accuracy increase from 45% to 79% (+34 percentage points). Additionally, refinements in the fine-tuning pipeline eliminated schema generation errors entirely, reducing their occurrence from 24% to 0%.

To contextualize our error profile, we compared error distributions across models (Table 6). While our model achieves lower overall error rates than the base Llama 3.2 3B (21% vs 55%), the composition of errors differs significantly. The key differentiator is the complete elimination of schema

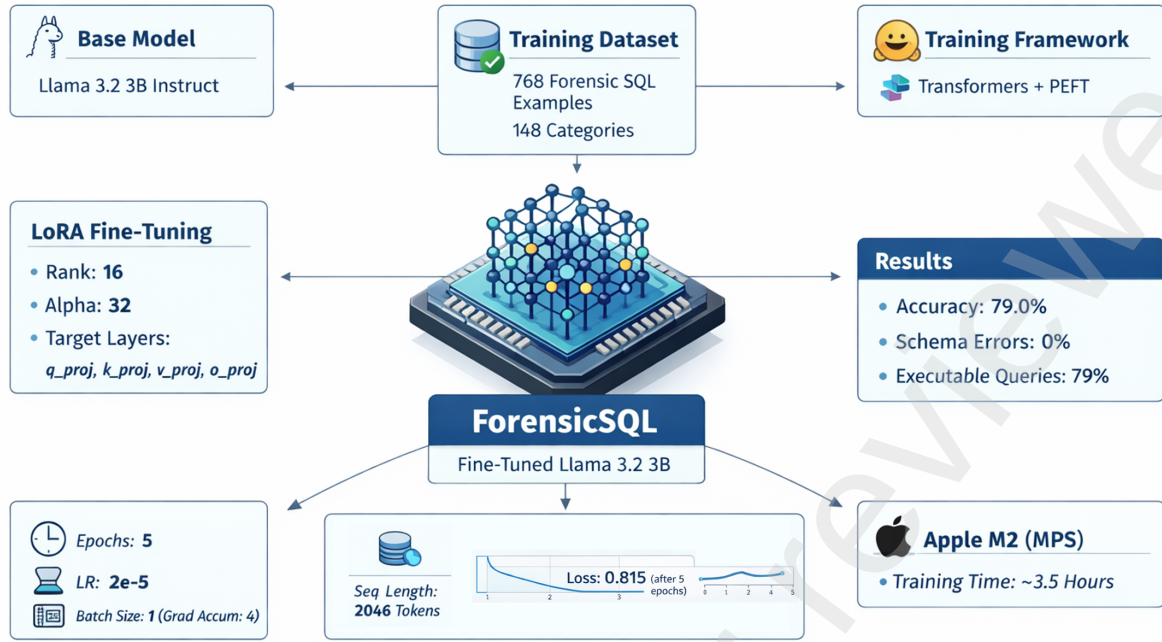


Figure 1: Parameters and Settings of the Fine-tuning process

Table 4
Comparison of ForSQLiteLM with other language models.

Model	Parameters	Training	Accuracy (%)
ForSQLiteLM (3B + LoRA)	3B	3.5h (M2)	79
Base LLaMA 3.2 3B	3B	—	45
CodeLLaMA 7B	7B	—	55
GPT4o	1.5T	—	87

Table 5
Execution Accuracy by Query Difficulty

Model	Easy(%)	Medium(%)	Hard(%)
ForSQLiteLM	94	81	62
Base LLaMA 3.2 3B	62	41	32
CodeLLaMA 7B	71	48	39
GPT4o	97	90	75

generation errors, which plagued both baseline models. This demonstrates that our training improvements specifically addressed a critical failure mode that affects general-purpose models when applied to this task. Interestingly, our column error rate (18%) is comparable to GPT4o (11%), despite the massive parameter difference (3B vs 1.5T). This suggests that column hallucination is an inherent challenge for autoregressive language models in Text-to-SQL tasks, regardless of model size. The similarity in error rates indicates that our fine-tuning successfully brought the small model to parity with large foundation models on this specific error dimension. Although cloud-based large language models such as GPT-4o currently achieve higher overall accuracy in Text-to-SQL tasks, the use of a locally deployed and self-trained model offers several practical advantages in the

forensic context. While GPT-4o reached an accuracy of approximately 87% compared to 79% achieved by the our fine-tuned model, this performance gap must be evaluated in light of operational, legal, and economic constraints commonly encountered in forensic laboratories.

From a cost perspective, the use of commercial cloud APIs introduces recurring expenses, which may amount to several hundred US dollars per year even for moderate query volumes. In contrast, ForensicSQL enables inference at effectively zero marginal cost once the model has been deployed locally.

Data privacy represents another critical factor. Cloud-based models require the transmission of potentially sensitive forensic data to external service providers, which may conflict with legal requirements, internal policies, or evidentiary chain-of-custody principles. Local inference eliminates this risk by ensuring that all data remains within the controlled forensic environment. Closely related is availability: a locally deployed model can be used offline and is therefore independent of network connectivity or third-party service availability, which is essential in secure or isolated laboratory settings.

Table 6
Error Distribution Across Models

Model	Syntax	Schema Gen	Column	Table	Total
ForSQLiteLM	3.0%	0.0%	18.0%	0.0%	21.0%
Llama 3.2 3B	15.0%	8.0%	23.0%	9.0%	55.0%
CodeLlama 7B	11.0%	9.0%	19.0%	6.0%	45.0%
GPT4o	0.0%	1.0%	11.0%	1.0%	13.0%

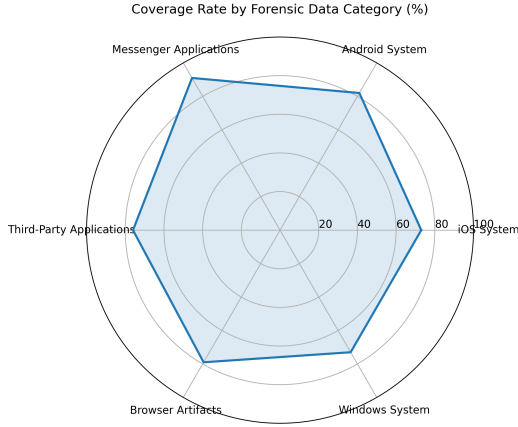


Figure 2: Benchmark results by category

Furthermore, a self-trained model allows for domain-specific customization through fine-tuning, enabling continuous adaptation to evolving forensic databases, schemas, and investigative practices. In contrast, commercial models typically operate as fixed black-box systems with limited transparency and no possibility for user-driven adaptation. Finally, the comparatively small model size of ForensicSQL, with approximately 3 billion parameters, allows it to run efficiently on commodity hardware, whereas state-of-the-art cloud models are assumed to rely on substantially larger architectures.

The spider diagram in Figure 2 shows how well the system can process different types of forensic data. It works best with messenger apps at 91% coverage – meaning it can analyze communication data very well. Android system data is also well supported at 82%. For third-party apps, coverage is 76%, while iOS and Windows system data both achieve 73%. Overall, the system works solidly across all platforms, with particular strengths in messenger apps and Android data.

Together, these findings demonstrate that careful dataset validation and execution-based evaluation are critical for achieving reliable performance in forensic Text-to-SQL systems.

4.3. Threats to Validity

Several limitations should be considered when interpreting the results of this study. First, dataset representativeness poses a potential threat to external validity. Although the dataset covers a broad range of forensic artifacts, platforms,

and schema complexities, it cannot fully capture the diversity of real-world mobile applications. Consequently, model performance may vary when applied to previously unseen applications or schema designs. Second, the evaluation focuses on execution correctness of generated SQL queries, which, while essential for forensic applicability, does not fully account for the investigative context in which queries are used. Some semantically imperfect queries may still yield partially useful results for human analysts, whereas strictly correct queries may omit contextual nuances relevant to an investigation. Third, query complexity distribution may influence performance metrics. Although difficulty levels are balanced, complex queries often involve multiple valid formulations. The evaluation framework considers only predefined reference queries, which may underestimate model performance in cases where alternative but semantically equivalent SQL formulations exist. Fourth, the model was fine-tuned and evaluated using a specific base architecture and quantization setup. While this reflects realistic deployment constraints on investigative hardware, results may not directly generalize to larger models or different fine-tuning strategies. However, this design choice aligns with the practical focus of this work on local, privacy-preserving forensic analysis.

5. Building a RAG pipeline for FQLite

Following the successful completion of the fine-tuning process, the model was integrated into FQLite², an open-source software tool designed for the forensic analysis of SQLite databases. One of FQLite's distinguishing capabilities is the forensic recovery of deleted SQLite databases, enabling investigators to examine data that is no longer accessible through intact or active database structures. FQLite is characterized by a high recovery rate.

With version 3.1, the tool was extended by an analysis component referred to as the *SQL-Analyzer*. This module allows investigators to execute SQL statements directly in order to examine the contents of recovered databases. However, effective use of this functionality requires substantial expertise in SQL and data analysis. The integration of AI-assisted analytical components is intended to significantly reduce this technical barrier. By providing natural language interfaces for complex queries, such assistants can simplify investigative workflows and enable even users with limited

²<https://github.com/pawlaszczyk/FQLite>

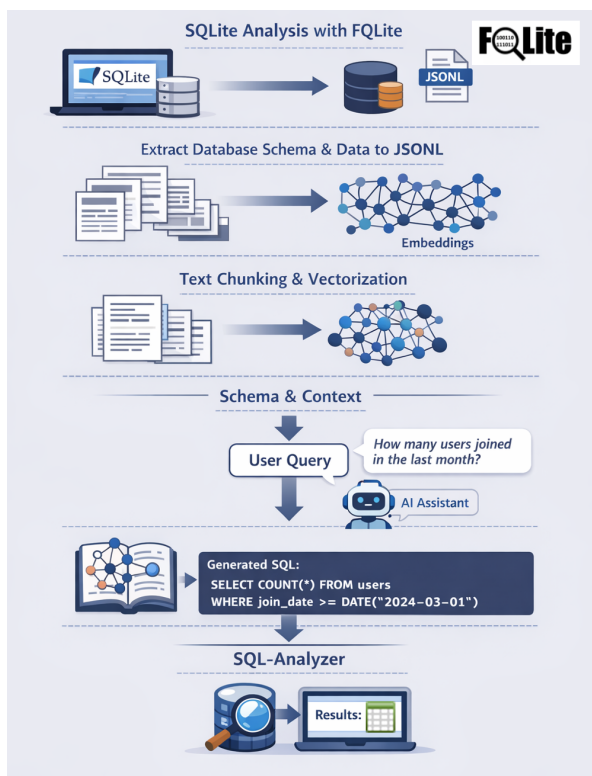


Figure 3: The FQLite RAG-pipeline for Text-to-SQL

technical training to conduct detailed and meaningful data analyses.

The integration of an AI-supported assistant is carried out via a special RAG pipeline. A Retrieval-Augmented Generation (RAG) pipeline combines a search component with a language model by first retrieving relevant documents or data from a knowledge base and then using this information to generate an answer. This ensures that the model's output is based on concrete, verifiable sources rather than only on its internal knowledge. The structure of the FQLite RAG pipeline is shown in Figure 3.

First, during the data collection and curation phase, the table schema and data records are extracted using FQLite. After this preprocessing step, embedding generation is started. Each text segment from our database schema is converted into a numerical vector using an embedding model. These embeddings capture semantic similarity rather than simple keyword overlaps. In this step, we use our self-trained LLM.

When a user makes a query, the pipeline enters the retrieval phase. The query is first converted into an embedding using the same embedding model. A similarity search is then performed on the vector index to identify the most relevant data. The generated embeddings are then stored in a vector index.

The retrieved chunks are then passed to a context assembly and prompt construction step. Here, the schema information is combined with the user query into a structured prompt. Finally, the generation phase uses the LLM to

produce an answer that is grounded in the retrieved material. The result is an SQL prompt. This can be executed by the user directly in the SQL Analyzer component of FQLite (Figure 4).

6. Discussion and Future Work

The results demonstrate that a comparatively small, locally deployable LLM can achieve strong performance on forensic Text-to-SQL tasks when trained on a carefully curated, domain-specific dataset. Despite comprising only 3 billion parameters, the proposed model attains an overall execution accuracy of 79%, with particularly high performance on easy and medium queries. This finding is notable given that many recent Text-to-SQL approaches rely on significantly larger models or cloud-based inference, which may be impractical in privacy-sensitive forensic environments. The observed performance highlights the importance of domain adaptation over model scale. By fine-tuning the model on authentic SQLite schemas and execution-validated forensic queries, the system learns task-relevant reasoning patterns that are not captured by general-purpose pre-training alone. The elimination of schema generation errors and the low rate of syntax errors further indicate that the model internalizes structural constraints effectively, even with limited parameter capacity. At the same time, the reduced accuracy on complex queries underscores inherent limitations of smaller models in multi-table reasoning and schema linking. Semantic errors related to missing joins or table confusion suggest that future improvements should focus on enhancing schema awareness rather than increasing model size alone. Hybrid approaches that combine LLM generation with explicit schema reasoning, constrained decoding, or retrieval-based schema context may offer promising directions. From a practical perspective, the ability to fine-tune and deploy the model on standard investigative hardware represents a significant advantage. The model's modest memory footprint, fast inference times, and local execution make it suitable for operational forensic workflows, where data confidentiality, reproducibility, and tool integration are critical. In this context, achieving near-state-of-the-art performance with a lightweight model presents a compelling trade-off between accuracy and deployability.

Future work will explore several avenues to further improve performance and robustness. These include incorporating explicit schema linking mechanisms, expanding coverage of complex forensic scenarios, and integrating human-in-the-loop validation to support investigator oversight. Additionally, evaluating the approach across a wider range of forensic tools and evolving application schemas will be essential to assess long-term generalizability. Overall, this study demonstrates that effective forensic Text-to-SQL systems do not necessarily require large-scale models, but rather benefit most from realistic datasets, execution-based evaluation, and deployment-aware design choices.

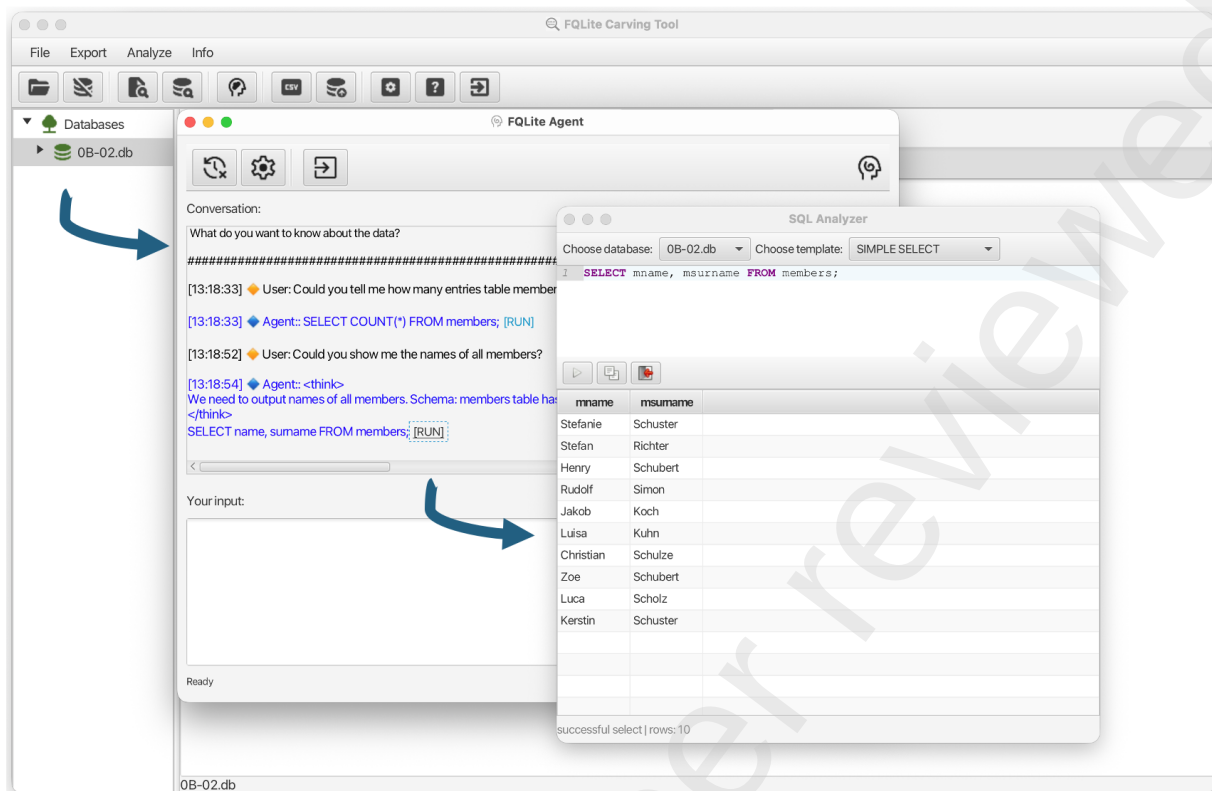


Figure 4: Integration of a Text-to-SQL RAG-Pipeline into FQLite

We would like to point out that all components of our approach presented in this article, the dataset, the fine-tuned model, and the FQLite toolset, are freely available.

7. Conclusion

This work presented a lightweight, domain-specific LLM for forensic Text-to-SQL generation, demonstrating that effective database analysis support can be achieved without large-scale models or cloud-based deployment. By fine-tuning a 3B-parameter model on an execution-validated forensic SQLite dataset, the proposed approach achieves strong performance across a wide range of investigative queries while remaining deployable on standard forensic hardware. The results highlight the importance of domain adaptation, realistic schemas, and execution-based evaluation over model size alone. Despite inherent challenges in complex multi-table reasoning, the model significantly reduces manual query formulation effort and supports privacy-preserving forensic workflows. Overall, this study underscores the viability of compact, locally deployable LLMs as practical tools for modern digital investigations.

Finally, it was demonstrated how a trained model can be integrated into an existing analysis tool via a RAG pipeline, making it directly usable for investigators. Furthermore, the proposed approach also offers great potential for training in mobile forensics.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is partially supported by the projects Inter-reg ***** 2021-2027, Project No. *****, Co-financed by the European Union.

CRediT authorship contribution statement

xxxxx xxxxxxxxxx: original draft, Visualization, Conceptualization and implementation of this Approach, Methodology, Software. *****: Writing - review & editing, Data curation. *****: implementation of Software, review & editing. ***: implementation of Software, review & editing. *****: implementation of Software, review & editing. *****: Writing, review & editing, Supervision. ****: Supervision.